Anexo 12B

Playwright - Visual Regression Testing

Curso: TypeScript

150 min

Objetivos de Aprendizaje

- Configurar Playwright con TypeScript
- Escribir tests E2E con selectores tipados
- Implementar Page Objects con tipos
- Implementar visual regression testing
- Testear aplicaciones SSR/Next.js
- Configurar CI/CD para testing

1. Por qué necesitamos tests E2E

10 minutos

La aplicación que en mi máquina funciona

```
// Tests unitarios NO detectan:
// - El botón de préstamo realmente funciona?
// - Se muestra el modal de confirmación?
// - La llamada API se hace correctamente?
// - Qué pasa si hay un error de red?
```

Problema: Los tests unitarios prueban partes aisladas

Solución: Tests E2E con Playwright

- Automatiza el navegador real
- Prueba flujos completos de usuario
- Detecta problemas de integración
- Verifica que la UI funciona

2. Playwright con TypeScript

30 minutos

¿Qué es Playwright?

Framework de testing E2E desarrollado por Microsoft.

- **Multi-navegador:** Chromium, Firefox, WebKit
- **Multi-plataforma:** Windows, Mac, Linux
- **Velocidad:** Ejecución rápida con paralelización
- **Autowaiting:** Espera automática de elementos
- **Grabación:** Genera tests con codegen
- **TypeScript:** Soporte nativo de tipos

Instalación

```
npm init playwright@latest
# O manualmente:
npm install -D @playwright/test
npx playwright install --with-deps chromium
```

Configuración Básica

```
// playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './e2e',
  fullyParallel: true,
  retries: process.env.CI ? 2 : 0,
  reporter: [['html']],
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
  ],
  webServer: {
    command: 'npm run dev',
    url: 'http://localhost:3000'
```

Lab: Configurar Playwright

```
// 1. Instalar Playwright
// npm init playwright@latest -- --lang=TypeScript
// npm install -D @playwright/test

// 2. playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './e2e',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
  ]
```

2.1. TypeScript Avanzado en Playwright

Fixtures y Matchers

Fixtures Tipadas

```
// e2e/fixtures/auth.fixture.ts
import { test as base } from '@playwright/test';

interface AuthFixtures {
  usuarioAutenticado: { id: string; nombre: string; token: string };
  hacerLogin: (usuario: string) => Promise<void>;
}

export const test = base.extend<AuthFixtures>({
  usuarioAutenticado: async ({ page }, use) => {
    await use({ id: '123', nombre: 'Bibliotecario', token: 'mock-token' });
  },
  hacerLogin: async ({ page }, use) => {
    await use(async (usuario: string) => {
      await page.goto('/login');
      await page.fill('[name="username"]', usuario);
      await page.click('[type="submit"]');
    });
  }
```

Matchers Personalizados

```
import { expect } from '@playwright/test';

expect.extend({
  async toHaveLibro(received: Locator, titulo: string) {
    const libro = received.locator('[data-testid="libro-card"]')
      .filter({ hasText: titulo });
    const count = await libro.count();

    if (count === 0) {
      return {
        pass: false,
        message: () => `Expected to find libro "${titulo}"`,
      };
    }
    return { pass: true, message: () => '' };
  },
});
```

Lab: Fixtures y Matchers Tipados

```
// e2e/fixtures/tasks.fixtures.ts
import { test as base, Page } from '@playwright/test';

interface TaskFixtures {
  mockApiTasks: (tasks: Task[]) => Promise<void>;
  paginaTareas: Page;
}

interface Task {
  id: string;
  title: string;
  status: 'pending' | 'completed';
}

export const test = base.extend<taskfixtures>({
  mockApiTasks: async ({ page }, use) => {
    await use(async (tasks: Task[]) => {
      await page.route('/api/tasks', route =>
        route.fulfill({ json: tasks })
```

3. Page Objects con Tipos

25 minutos

¿Qué son los Page Objects?

Patrón de diseño que encapsula la interacción con páginas web.

- **Mantenibilidad:** Si cambia la UI, solo se actualiza el Page Object
- **Reutilización:** Métodos reutilizables
- **Legibilidad:** Tests más limpios
- **Tipado:** TypeScript asegura selectores correctos

Estructura de Page Objects

```
// e2e/pages/ListadoLibrosPage.ts
import { Page, Locator } from '@playwright/test';

export class ListadoLibrosPage {
  private page: Page;
  private libroCards: Locator;
  private botonPrestar: Locator;

  constructor(page: Page) {
    this.page = page;
    this.libroCards = page.locator('[data-testid="libro-card"]');
    this.botonPrestar = page.locator('[data-testid="boton-prestar"]');
  }

  async goto(): Promise<void> {
    await this.page.goto('/libros');
  }

  async clickPrestar(titulo: string): Promise<void> {
```

Selectores Recomendados

```
// Por testId (recomendado)
page.locator('[data-testid="boton-prestar"]');

// Por rol (accesibilidad)
page.getByRole('button', { name: 'Prestar' });

// Por label
page.getByLabel('Nombre del libro');
```

Lab: Page Objects para el Caso 1r

```
// e2e/pages/TareasPage.ts
export class TareasPage {
  constructor(private page: Page) {}

  async goto(): Promise<void> {
    await this.page.goto('/tareas');
  }

  async agregarTarea(titulo: string): Promise<void> {
    await this.page.fill('[data-testid="input-tarea"]', titulo);
    await this.page.click('[data-testid="btn-agregar"]');
  }

  async completarTarea(titulo: string): Promise<void> {
    const tarea = this.page.locator('[data-testid="tarea"]').filter({ hasText: titulo });
    await tarea.locator('[data-testid="btn-completar"]').click();
  }
}
```

4. Visual Regression Testing

30 minutos

¿Qué es Visual Regression Testing?

Compara screenshots antes y después de cambios para detectar diferencias visuales.

- **Detectar cambios inesperados:** CSS, layout, colores
- **UI consistente:** Sin regresiones visuales
- **Múltiples configuraciones:** Temas, viewports
- **Automatización:** Sin revisión manual

Flujo de Trabajo

1. **Baseline:** Primera ejecución genera screenshots de referencia
2. **Comparación:** Ejecuciones siguientes comparan con baseline
3. **Diff:** Si hay diferencias, se generan imágenes
4. **Threshold:** Tolerancia a diferencias menores

Configuración

```
// playwright.config.ts
export default defineConfig({
  use: {
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
});

// Test de regression visual
test('la página de libros no ha cambiado', async ({ page }) => {
  await page.goto('/libros');
  await expect(page).toHaveScreenshot('listado-libros.png');
});
```

Comando: `npx playwright test --update-snapshots`

Visual Regression con Theming

```
// Configuración para light/dark
export default defineConfig({
  projects: [
    { name: 'light', use: { colorScheme: 'light' } },
    { name: 'dark', use: { colorScheme: 'dark' } },
  ],
});

// Test de dark mode
test('libros en modo oscuro', async ({ page }) => {
  await page.emulateMedia({ colorScheme: 'dark' });
  await page.goto('/libros');
  await expect(page).toHaveScreenshot('libros-dark.png');
});
```

Visual Regression Mobile

```
// Configuración mobile
export default defineConfig({
  projects: [
    { name: 'Mobile', use: { viewport: { width: 375, height: 667 } } },
    { name: 'Desktop', use: { viewport: { width: 1280, height: 720 } } },
  ],
});

// Test responsive
test('layout responsive', async ({ page }) => {
  await page.setViewportSize({ width: 375, height: 667 });
  await page.goto('/libros');
  await expect(page).toHaveScreenshot('libros-mobile.png');
});
```

Lab: Visual Regression

```
// 1. Habilitar screenshots en playwright.config.ts
export default defineConfig({
  use: { screenshot: 'only-on-failure' },
});

// 2. Generar baseline
// npx playwright test --update-snapshots

// 3. Test de regression visual
test('tareas - baseline', async ({ page }) => {
  await page.goto('/tareas');
  await expect(page).toHaveScreenshot('tareas-list.png');
});

// 4. Test con theming
test('tareas - dark mode', async ({ page }) => {
  await page.emulateMedia({ colorScheme: 'dark' });
  await page.goto('/tareas');
  await expect(page).toHaveScreenshot('tareas-dark.png');
```

4.1. Mocking de Red

15 minutos

Interceptando Peticiones

Permite capturar, modificar o simular respuestas HTTP.

- **Aislamiento:** No depende de APIs externas
- **Determinismo:** Resultados predecibles
- **Velocidad:** Sin latencia de red
- **Escenarios:** Simular errores

Stubbing Básico

```
// Stubear respuesta de API
await page.route('/api/libros', async route => {
  await route.fulfill({
    status: 200,
    json: [
      { id: '1', titulo: 'TypeScript Básico', disponible: true },
      { id: '2', titulo: 'TypeScript Avanzado', disponible: false }
    ]
  });
});

// Simular error de red
await page.route('/api/libros', async route => {
  await route.abort('failed');
});
```

Lab: Stubbing de API

```
// e2e/utils/mock-api.ts
export class MockApi {
  constructor(private page: Page) {}

  async stubLibros(libros: Libro[]) {
    await this.page.route('/api/libros', route =>
      route.fulfill({ status: 200, json: libros })
    );
  }

  async stubErrorRed() {
    await this.page.route('/api/**', route => route.abort('failed'));
  }
}

// e2e/fixtures/api.fixture.ts
export const test = base.extend<{ mockApi: MockApi }>({
  mockApi: async ({ page }, use) => await use(new MockApi(page)),
});
```

5. Testing SSR/Next.js

25 minutos

¿Por qué es diferente el testing SSR?

- **Contenido en HTML inicial:** El servidor envía HTML con datos
- **Hydration:** React "hidrata" el HTML en el cliente
- **Streaming:** Contenido llega progresivamente
- **Cookies/Sessions:** Gestión de autenticación diferente

Esperar Hydration

```
// Esperar a que React hydrate
await page.waitForLoadState('networkidle');

// Verificar HTML inicial
test('contenido SSR presente', async ({ page }) => {
  await page.goto('/libros');
  const html = await page.content();
  expect(html).toContain('Biblioteca');
});
```

Testing de API Routes

```
// Test de API
test('el endpoint de libros funciona', async ({ request }) => {
  const response = await request.get('/api/libros');
  expect(response.ok()).toBeTruthy();

  const libros = await response.json();
  expect(libros).toHaveLength(3);
});
```

Testing de Streaming SSR

```
// Test de streaming
test('contenido se carga progresivamente', async ({ page }) => {
  await page.goto('/libros');

  // Verificar título inmediato
  await expect(page.locator('h1')).toContainText('Biblioteca', { timeout: 5000 });

  // Verificar carga de libros
  await expect(page.locator('[data-testid="libro-card"]').first())
    .toBeVisible({ timeout: 10000 });
});
```

Testing de Autenticación

```
// Test con cookie de sesión
test('usuario autenticado puede prestar libros', async ({ page }) => {
  await page.context().addCookies([{
    name: 'next-auth.session-token',
    value: 'mock-session-token',
    domain: 'localhost',
    path: '/',
  }]);

  await page.goto('/prestamos');
  await expect(page.locator('[data-testid="boton-prestar"]').first()).toBeEnabled();
});
```

Lab: Testear Next.js del Caso 1r

```
// Test 1: Contenido SSR
test('contenido SSR presente', async ({ page }) => {
  await page.goto('/tareas');
  const html = await page.content();
  expect(html).toContain('Tareas');
});

// Test 2: API Routes
test('API de tareas funciona', async ({ request }) => {
  const response = await request.get('/api/tasks');
  expect(response.ok()).toBeTruthy();
});

// Test 3: Streaming SSR
test('contenido carga progresivamente', async ({ page }) => {
  await page.goto('/tareas');
  await expect(page.locator('h1')).toContainText('Tareas', { timeout: 5000 });
  await expect(page.locator('[data-testid="task"]').first()).toBeVisible({ timeout: 15000 });
});
```

Lab: Testing Avanzado de SSR

```
// Test: ISR (Incremental Static Regeneration)
test('página se regenera correctamente', async ({ request }) => {
  const response = await request.get('/api/revalidate?path=/tareas');
  expect(response.ok()).toBeTruthy();
});

// Test: Streaming con Suspense
test('muestra fallback mientras carga', async ({ page }) => {
  await page.goto('/tareas');
  const skeleton = page.locator('[data-testid="loading-skeleton"]');
  await expect(skeleton).toBeVisible();
  await expect(skeleton).toBeHidden({ timeout: 10000 });
});

// Test: Login y persistencia de sesión
test('login y persistencia', async ({ page }) => {
  await page.goto('/login');
  await page.fill('[name="email"]', 'user@test.com');
  await page.fill('[name="password"]', 'password123');
```

6. Ejercicio Integrador

Tests E2E completos

Objetivo

Implementar suite completa de tests E2E para el sistema de biblioteca

- Tests para todos los flujos principales
- Page Objects para todas las páginas
- Al menos un test de visual regression
- Tests de API routes

Flujos a Testear

1. Listado de libros
2. Búsqueda de libros
3. Préstamo de libros
4. Devolución de libros
5. Registro de socios

Resumen

Puntos Clave

- Playwright = framework E2E moderno de Microsoft
- Fixtures tipadas = reutilización y tipos estrictos
- Page Objects = mantenibilidad y legibilidad
- Visual regression = detección automática de cambios UI
- page.route() = mocking de red para tests deterministas
- SSR testing = esperar hydration y streaming
- Cookies = manejar autenticación en tests

Referencias

Recursos Adicionales

- Playwright Documentation
- Playwright API
- Test Configuration
- Visual Regression Guide
- Testing Next.js

axiateam
RDT company

Gracias!

Anexo 12B Completado

Fin del curso TypeScript